

1.2 ROS 2 架构概览

所属阶段： 第一阶段 · ROS 2 入门与环境搭建

讲次数量： 3 节

预计时长： ~20 分钟

前置依赖： 1.1 为什么需要机器人操作系统

学习目标

完成本节后，学员将能够：

- 描述 ROS 2 的五层架构及其各层职责
- 理解 rcl、rclpy、rclcpp、rmw 和 DDS 之间的关系
- 根据项目需求合理选择 Python 或 C++ 作为开发语言
- 制定 ROS 2 发行版选择策略，了解 Humble/Jazzy 差异，并知道 2026 年最新 LTS 为 Lyrical
- 明确本课程的技术栈选型：Jazzy 为主、Humble 为辅

讲次 1：ROS 2 的分层架构

知识点

ROS 2 由多层库和接口组成。为了便于初学者理解，本课程把常见的数据路径归纳为下面五层；这是**教学模型**，不是官方要求所有实现都严格遵循的唯一分层方式。



第 1 层：DDS 实现（底层中间件）

DDS (Data Distribution Service, 数据分发服务) 是 Jazzy 默认通信路径中的底层中间件，负责：

- 网络传输 (UDP 多播、TCP 单播、共享内存等，视环境和配置而定)
- 服务发现 (SPDP/SEDP 协议)
- QoS 策略的执行

ROS 2 不绑定单一 DDS 厂商，而是通过 RMW 接口支持多种实现。除下列 DDS 实现外，ROS 2 也在发展 Zenoh 等非 DDS RMW，因此“RMW 下方一定是 DDS”只适用于本课程使用的默认路径。

DDS 实现	包名	特点
eProsima Fast DDS	rmw_fastrtps_cpp	Jazzy 默认实现，社区最活跃
Eclipse Cyclone DDS	rmw_cyclonedds_cpp	开源实现，常用于 Linux 机器人系统
RTI Connex DDS	rmw_connextdds	商业级，功能最全，需许可证

切换 DDS 实现只需设置环境变量：

```
# 查看当前使用的 RMW 实现（推荐方式）
ros2 doctor --report 2>/dev/null | grep "middleware name"
# 输出: middleware name      : rmw_fastrtps_cpp

# 也可查看环境变量（默认未设置，表示使用系统默认 Fast DDS）
echo $RMW_IMPLEMENTATION # （Windows 用户使用 `echo %RMW_IMPLEMENTATION%`）
# 输出为空 = 使用默认 rmw_fastrtps_cpp

# 切换到 Cyclone DDS
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
```

作为应用开发者，你通常不需要直接操作 DDS 层，但了解它的存在有助于排查通信问题和优化性能。

第 2 层: RMW (ROS Middleware Interface)

RMW 是 ROS 2 对底层中间件的抽象接口层。它定义统一的 C API，使上层代码不必绑定某一家 DDS 或其他中间件实现。

RMW 层的主要职责：

- 将 Node、Publisher、Subscriber、Service 等 ROS 2 通信概念映射到底层实体；Action 在 ROS 2 上层由话题和服务等机制组合实现，并不是一个独立 DDS 实体
- 管理 DDS 域 (Domain ID)，默认为 Domain 0
- 提供统一的 QoS 配置接口

```
# 查看已安装的 RMW 实现
ros2 doctor --report | grep -i rmw
```

第 3 层: RCL (ROS Client Library — 核心层)

RCL 是 ROS 2 的**语言无关核心库**，用 C 语言实现。它提供：

- 节点生命周期管理（创建、销毁）
- 发布者/订阅者/服务/动作/参数的底层 API
- 时钟与时间源管理
- 日志系统（`rcutils`）

RCL 之上还有 `rcl_action`（动作库）和 `rcl_lifecycle`（生命周期节点库）等扩展模块。

为什么用 C 实现核心层？

- C 语言性能高、内存可控，适合实时场景
- 可以被任何语言绑定（Python、C++、Rust、Java 等）
- 避免重复实现核心逻辑

第 4 层: 客户端库 (Client Libraries)

客户端库将 RCL 的 C API 封装为各语言友好的接口：

rclcpp (C++ 客户端库)

```

#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"

class MinimalPublisher : public rclcpp::Node {
public:
    MinimalPublisher() : Node("minimal_publisher") {
        publisher_ = this->create_publisher<std_msgs::msg::String>("topic", 10);
        timer_ = this->create_wall_timer(
            std::chrono::milliseconds(500),
            std::bind(&MinimalPublisher::timer_callback, this));
    }
private:
    void timer_callback()
    {
        auto msg = std_msgs::msg::String();
        msg.data = "Hello, ROS 2! " + std::to_string(count_++);
        publisher_->publish(msg);
        RCLCPP_INFO(this->get_logger(), "Publishing: '%s'", msg.data.c_str());
    }
    rclcpp::Publisher<std_msgs::msg::String>::SharedPtr publisher_;
    rclcpp::TimerBase::SharedPtr timer_;
    size_t count_ = 0;
};

```

rclpy (Python 客户端库)

```

import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class MinimalPublisher(Node):
    def __init__(self):
        super().__init__('minimal_publisher')
        self.publisher_ = self.create_publisher(String, 'topic', 10)
        self.timer = self.create_timer(0.5, self.timer_callback)
        self.count = 0

    def timer_callback(self):
        msg = String()
        msg.data = f'Hello, ROS 2! {self.count}'
        self.count += 1
        self.publisher_.publish(msg)
        self.get_logger().info(f'Publishing: '{msg.data}''')

```

两者共享 Node、Publisher、Subscription、Service 等概念，但函数签名、执行器、生命周期与资源管理方式并不完全相同。理解通信模型后迁移会更快，仍需阅读对应客户端库 API。

其他客户端库（本课程不深入）：

库	语言	说明
rcl	C	微控制器/嵌入式
rclrs	Rust	内存安全的高性能开发
ros2_java	Java	Android 机器人应用
ros2_dotnet	C#	Windows/.NET 生态

第 5 层：应用层（你的代码）

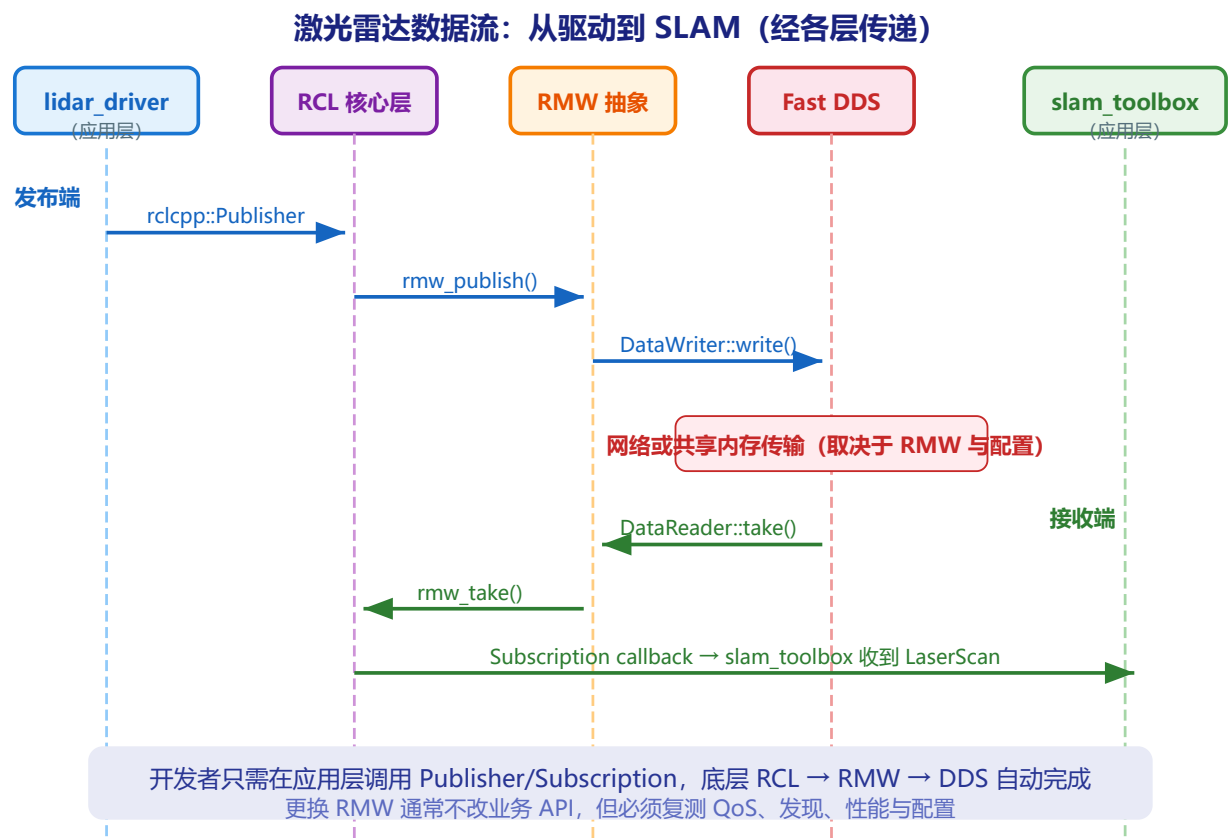
应用层包含你实际编写的所有 ROS 2 软件：

- **节点 (Node)**：ROS 图中的逻辑通信参与者；一个进程可以包含一个或多个节点，节点不等同于进程
- **软件包 (Package)**：节点的组织单元，包含代码、配置、启动文件
- **Launch 文件**：批量启动多个节点的配置文件
- **接口定义 (.msg / .srv / .action)**：自定义消息类型
- **URDF/Xacro**：机器人模型描述

应用层通过客户端库（rclpy/rclcpp）与底层通信，无需直接接触 RCL 或 DDS。

数据流示例

以"激光雷达数据从驱动到 SLAM"为例，数据流经各层的过程：



图：激光雷达数据从 lidar_driver 经 RCL → RMW → DDS 传递到 slam_toolbox 的完整时序

该图是概念上的简化路径：发布端和订阅端各自都有客户端库、RCL 与 RMW 调用，底层也可能走进程内通信或共享内存，而不是每条消息都经过 UDP 网络。它用于解释分层职责，不用于推导精确调用栈或性能。

关键点

- 五层图是课程教学模型；RMW 下方通常是 DDS，也可以连接 Zenoh 等非 DDS 实现
 - 应用开发主要使用 rclpy/rclcpp；更换 RMW 通常不改业务 API，但必须复测发现、QoS、配置和性能
-

讲次 2：客户端库生态——Python vs C++ 的选择

知识点

ROS 2 同时提供 Python (rclpy) 和 C++ (rclcpp) 客户端库。本课程在通信基础阶段提供两种语言的对照，项目阶段按性能、依赖生态和维护成本选择主实现，不机械复制两套代码。

语言特性对比

维度	Python (rclpy)	C++ (rclcpp)
开发速度	快，代码量少	较慢，需要更多模板代码
运行性能	适合多数编排与中低频节点；纯 Python CPU 密集计算受 GIL 影响	适合低延迟、高吞吐和资源受限路径
内存管理	自动 GC	手动/智能指针管理
实时性	不作为确定性实时控制首选	更便于做实时工程，但仍需 PREEMPT_RT、内存/调度设计和实测验证
调试便利性	高，REPL 交互式调试	中等，需编译后调试
生态集成	ML/CV 库丰富 (PyTorch、OpenCV)	性能库丰富 (Eigen、PCL)
类型安全	动态类型，运行时检查	静态类型，编译期检查
部署体积	需要 Python 运行时和依赖包	编译为本地二进制，但通常仍动态依赖 ROS 2 与系统库

何时选择 Python

Python 常用于 ROS 2 的**快速原型和算法验证**：

1. 原型开发与概念验证 (Prototyping)

```
# 最小发布者示例—用于快速验证想法
import rclpy
from rclpy.executors import ExternalShutdownException
from rclpy.node import Node
from std_msgs.msg import String

class QuickTest(Node):
    def __init__(self):
        super().__init__('quick_test')
        self.pub = self.create_publisher(String, 'test', 10)
        self.create_timer(1.0, lambda: self.pub.publish(String(data='hello')))

def main():
    rclpy.init()
    node = QuickTest()
    try:
        rclpy.spin(node)
    except (KeyboardInterrupt, ExternalShutdownException):
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == '__main__':
    main()
```

2. 数据处理与分析

- 激光雷达点云处理（结合 NumPy、Open3D）
- 图像处理与目标检测（结合 OpenCV、YOLO/Ultralytics）
- 传感器数据可视化与日志分析

3. 机器学习集成

- 深度学习模型推理（PyTorch、TensorFlow、ONNX Runtime）
- 强化学习训练环境接口
- 自然语言处理（语音交互、大模型指令理解）

4. 教学和脚本工具

- Launch 文件（Python 版 Launch 更灵活）
- 测试脚本、自动化工具
- 参数调优和实验脚本

何时选择 C++

C++ 常用于**性能敏感、资源受限和需要精细控制运行时行为**的场景：

1. 性能关键路径

- 高频传感器驱动节点（>100Hz 激光雷达数据处理 —— 注意：这里的“驱动”是用户空间的 ROS 2 节点，通过厂商 SDK 与硬件交互，而非 Linux 内核驱动）

- 对延迟和抖动敏感的控制节点；真正的高速伺服内环往往仍运行在 MCU、实时控制器或专用驱动器中
- 大规模点云处理（PCL 库原生 C++）

2. 实时系统

- 工业机械臂上层轨迹与控制接口；亚毫秒级内环通常由专用实时控制器承担
- 自动驾驶规划与控制模块
- 需要 PREEMPT_RT 等实时系统能力的确定性场景；使用 C++ 本身不等于获得硬实时

3. 生产部署

- 嵌入式平台（NVIDIA Jetson、ARM 板）资源有限
- 希望减少解释器依赖，并更精细地控制内存和启动行为
- 长期运行的无人值守系统

4. 与现有 C/C++ 库集成

- 调用硬件厂商提供的 C/C++ SDK（如 Intel librealsense、Velodyne SDK），封装为 ROS 2 驱动节点
- 集成 Eigen（线性代数）、PCL（点云库）、OMPL（运动规划）

混合开发策略（推荐）

实际项目中，Python 和 C++ 节点可以共存于同一 ROS 2 系统，通过话题/服务/动作通信，互不干扰：
两种语言只要使用一致的 ROS 2 接口定义，就能通过话题、服务和动作协作。语言规划描述的是课程当前代码分布，不是“某类模块必须使用某种语言”的规则。

本课程的语言规划

阶段	Python 重点	C++ 重点
第一阶段	环境搭建、工具使用	环境搭建、工具使用
第二阶段	节点、话题、服务、动作	节点、话题、服务、动作
第三阶段	URDF 工具链	URDF 解析、TF 计算
第四阶段	Launch、配置与辅助脚本	仿真、ros2_control、MoveIt 2 示例
第五阶段	Nav2 配置、探索与巡逻工具	复用 Nav2/SLAM 现成组件
第六阶段	算法验证与测试工具	A* 规划器、Pure Pursuit 控制器插件
第七-八阶段	传感器处理、YOLO、语音与任务编排	按硬件接口和性能需求补充 C++ 能力

关键点

- 选型先看最坏时延、数据规模、依赖生态、部署资源和团队维护能力，再决定 Python、C++ 或混合方案
 - C++ 便于实时工程，但语言选择本身不提供硬实时保证；Python 也不等同于只能写演示代码
-

讲次 3：ROS 2 发行版选择策略

知识点

ROS 2 采用与 Ubuntu 类似的**定期发行版 (Distribution)** 模式。每个发行版有明确的发布日期、支持周期和对应的 Ubuntu 版本。选择合适的发行版对项目的长期维护至关重要。

时效性说明 (2026-07)：官方最新稳定版是 **Lyrical Luth (2026 年 5 月发布, LTS 支持至 2031 年 5 月)**，Kilted Kaiju 发布于 2025 年。课程没有追逐最新版，而是继续固定在已经完成 Stage1–8 构建和仿真验证的 **Jazzy**。Jazzy 仍处于官方支持期（至 2029 年 5 月），因此这是“经过验证的课程基线”，不是“当前最新发行版”。发行状态以 [ROS 2 官方 Releases](#) 为准。

本课程关注的两个发行版

ROS 2 Humble Hawksbill (LTS 长期支持版)

属性	值
发布日期	2022 年 5 月
支持截止	2027 年 5 月
支持平台	Ubuntu 22.04 (Jammy)
定位	长期支持版 (LTS)，适合已有 Ubuntu 22.04 项目
特点	存量生态较成熟，具体第三方包仍需逐项确认

```
# Humble 安装 (Ubuntu 22.04)
sudo apt install ros-humble-desktop
source /opt/ros/humble/setup.bash
```

ROS 2 Jazzy Jalisco (本课程验证基线, LTS)

属性	值
发布日期	2024 年 5 月
支持截止	2029 年 5 月 (LTS 长期支持版)
支持平台	Ubuntu 24.04 (Noble)
定位	仍受官方支持的 LTS，本课程全链路验证版本
特点	搭配 Ubuntu 24.04, Gazebo Harmonic, Python 3.12

```
# Jazzy 安装 (Ubuntu 24.04)
sudo apt install ros-jazzy-desktop
source /opt/ros/jazzy/setup.bash
```

Humble vs Jazzy 关键差异

对比项	Humble (LTS)	Jazzy (课程基线 LTS)
Ubuntu 版本	22.04 Jammy	24.04 Noble
Python 版本	3.10	3.12
Gazebo 版本	Gazebo Fortress (默认)	Gazebo Harmonic (默认)
包数量	最多 (生态最成熟)	快速增长中
企业采用	大量存量部署	新旧项目均有, 需按依赖和生命周期选型
课程角色	备选/兼容参考	主要教学版本

发行版选择策略

选择 Jazzy 的场景 (本课程默认) :

- 跟随本课程学习并复现实验
- 使用 Ubuntu 24.04 开发环境
- 需要课程已验证的 Gazebo Harmonic 仿真链路
- 需要使用 Jazzy 新引入的特性

选择 Humble 的场景:

- 企业生产环境 (LTS 保障到 2027)
- 现有项目基于 Ubuntu 22.04
- 依赖的第三方包尚未移植到 Jazzy
- 团队已大规模部署 Humble

选择 Rolling 的场景:

- ROS 2 核心开发者
- 需要测试最新未发布特性
- 不推荐用于学习和生产

本课程的双版本策略

本课程以 **Jazzy + Ubuntu 24.04** 作为主要教学环境, 同时提供 **Humble + Ubuntu 22.04** 的备选方案:

主要路径: Ubuntu 24.04 + ROS 2 Jazzy (推荐)

- └─ 所有 LAB 实验基于此环境
- └─ 所有代码示例优先 Jazzy 测试
- └─ 新版本特性 (Gazebo Harmonic 等) 仅 Jazzy 讲解

备选路径: Ubuntu 22.04 + ROS 2 Humble

- └─ 安装步骤平行提供
- └─ 核心概念相近, 但接口、参数和依赖仍需核对
- └─ 差异点会特别标注 (如 Gazebo 版本)

重要提示: ROS 2 的节点、话题、服务、动作和参数等核心概念跨发行版高度一致, 但 Gazebo、Nav2 参数、插件接口、Python/C++ 版本和第三方包可能不同。迁移时应重新安装依赖、阅读发行说明并完整回归测试, 不能只替换发行版名称。

查看当前环境的发行版

```
# 方法 1: 查看环境变量
echo $ROS_DISTRO
# 输出: jazzy

# 方法 2: 使用 ros2 命令
ros2 doctor --report | grep "distribution name"
# 输出: distribution name      : jazzy

# 方法 3: 查看已安装的包
apt list --installed 2>/dev/null | grep "ros-jazzy" | head -5
```

关键点

- 2026 年最新 LTS 是 Lyrical; Jazzy 仍受支持, 并因完成课程全链路验证而作为主基线
 - 发行版迁移必须复核操作系统、Gazebo、Nav2、插件 API、Python/C++ 依赖并完整回归, 不能预设迁移成本很低
-

本节总结

架构层次速查

层次	名称	职责	开发者是否直接接触
第 5 层	应用层	节点、Launch、URDF	是（主要工作层）
第 4 层	客户端库	rclpy / rclcpp	是（API 调用层）
第 3 层	RCL	语言无关核心	通常不直接接触
第 2 层	RMW	中间件抽象	通常不直接接触
第 1 层	传输实现（通常为 DDS）	网络通信执行	通常不直接接触

课程基线是 Ubuntu 24.04 + Jazzy + 默认 Fast DDS RMW；这是一组经过实测的起点，不限制项目按需求更换语言或 RMW。更换后需重新验证发现、QoS 和性能。

下一步

下一节 **1.3 开发环境搭建** 中，我们将动手配置完整的 ROS 2 开发环境：从 Ubuntu 24.04 虚拟机安装，到 ROS 2 Jazzy 部署，再到 VS Code 配置和第一个工作空间的创建。这是整个课程中实操性最强的章节，请准备好你的电脑。